refinement leads to design representations at much lower levels of abstraction. These can still be traced to requirements, but the connections may not be obvious at these lower levels of abstraction.

9.2.1 Software Quality Guidelines and Attributes

Throughout the design process, the quality of the evolving design is assessed with a series of technical reviews discussed in Chapter 16. McGlaughlin [McG91] suggests three characteristics that serve as a guide for the evaluation of a good design:

- The design should implement all explicit requirements contained in the requirements model, and it must accommodate all the implicit requirements desired by stakeholders.
- The design should be a readable, understandable guide for those who generate code and for those who test and subsequently support the software.
- The design should provide a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation perspective.

Each of these characteristics is a goal of the design process. But how is each of these goals achieved?

Quality Guidelines. To evaluate the quality of a design representation, you and other members of the software team must establish technical criteria for good design. In Section 9.3, we discuss design concepts that also serve as software quality criteria. For the time being, consider the following guidelines:

1. A design should exhibit an architecture that (a) has been created using recognizable architectural styles or patterns, (b) is composed of components that exhibit good design characteristics (these are discussed later in this chapter), and (c) can be implemented in an evolutionary fashion,[2] thereby facilitating implementation and testing.
2. A design should be modular; that is, the software should be logically partitioned into elements or subsystems.
3. A design should contain distinct representations of data, architecture, interfaces, and components.
4. A design should lead to data structures that are appropriate for the classes to be implemented and are drawn from recognizable data patterns.
5. A design should lead to components that exhibit independent functional characteristics.
6. A design should lead to interfaces that reduce the complexity of connections between components and with the external environment.
7. A design should be derived using a repeatable method that is driven by information obtained during software requirements analysis.
8. A design should be represented using a notation that effectively communicates its meaning.

2 For smaller systems, design can sometimes be developed linearly.

Chance alone will not achieve these design guidelines. They are achieved through the application of fundamental design principles, systematic methodology, and thorough review.

INFO



Assessing Design Quality—The Technical Review

Design is important because it allows a software team to assess the quality[3] of the software before it is implemented—at a time when errors, omissions, or inconsistencies are easy and inexpensive to correct. But how do we assess quality during design? The software can't be tested, because there is no executable software to test. What to do?

During design, quality is assessed by conducting a series of technical reviews (TRs). TRs are discussed in detail in Chapter 16,[4] but it's worth providing a summary of the technique at this point. A technical review is a meeting conducted by members of the software team. Usually two, three, or four people participate depending on the scope of the design information to be reviewed. Each person plays a role. The *review leader* plans the meeting, sets an agenda, and runs the meeting; the *recorder* takes notes so that nothing is missed; and the *producer* is the person whose work product (e.g., the design of a software component) is being reviewed. Prior to the meeting, each person on the review team is given a copy of the design work product and is asked to read it, looking for errors, omissions, or ambiguity. When the meeting commences, the intent is to note all problems with the work product so that they can be corrected before implementation begins. The TR typically lasts between 60 to 90 minutes. After the TR concludes, the review team determines whether further actions are required from the producer before the design work product can be approved as part of the final design model.

9.2.2 The Evolution of Software Design

The evolution of software design is a continuing process that has now spanned more than six decades. Early design work concentrated on criteria for the development of modular programs [Den73] and methods for refining software structures in a top-down "structured" manner ([Wir71], [Dah72], [Mil72]). Newer design approaches (e.g., [Jac92], [Gam95]) proposed an object-oriented approach to design derivation. More recent emphasis in software design has been on software architecture [Kru06] and the design patterns that can be used to implement software architectures and lower levels of design abstractions (e.g., [Hol06], [Sha05]). There is a growing emphasis on aspect-oriented methods (e.g., [Cla05], [Jac04]), model-driven development [Sch06], and test-driven development [Ast04], which focus on techniques for achieving more effective modularity and architectural structure in the designs that are created.

In the past 10 years, Search-Based Software Engineering (SBSE) techniques have been applied to all phases of the software engineering life cycle, including design [Har12]. SBSE attempts to solve software engineering problems using automated search techniques augmented by operations research and machine learning algorithms to provide design recommendations to software developers. Many modern software systems must accommodate a high degree of variability, both in their deployment environments and

3 The quality factors discussed in Chapter 23 can assist the review team as it assesses quality.

4 You might consider looking ahead to Chapter 16 at this time. Technical reviews are a critical part of the design process and are an important mechanism for achieving design quality.

the number of usage scenarios they expected to satisfy. Design of *variability-intensive systems*[5] requires developers to anticipate future changes in the features to be modified in future versions of the product being designed today [Gal16]. A detailed discussion of the design of variability-intensive systems is beyond the scope of this book.

Several design methods, growing out of the work just noted, are being applied throughout the industry. Like the analysis methods presented in Chapter 8, each software design method introduces unique heuristics and notation, as well as a somewhat parochial view of what characterizes design quality. Yet, each of these methods has common characteristics: (1) a mechanism for the translation of the requirements model into a design representation, (2) a notation for representing functional components and their interfaces, (3) heuristics for refinement and partitioning, and (4) guidelines for quality assessment.

Regardless of the design method that is used, you should apply a set of basic concepts to data, architectural, interface, and component-level design. These concepts are considered in the sections that follow.

TASK SET



Generic Task Set for Design

Please note: These tasks are often performed iteratively and in parallel. They are rarely completed sequentially and in isolation from one another unless you are following the waterfall process model.

1. Examine the information model, and design appropriate data structures for data objects and their attributes.
2. Using the analysis model, select an architectural style (pattern) that is appropriate for the software.
3. Partition the analysis model into design subsystems and allocate these subsystems within the architecture:
   - Be certain that each subsystem is functionally cohesive.
   - Design subsystem interfaces.
   - Allocate analysis classes or functions to each subsystem.
4. Create a set of design classes or components:
   - Translate an analysis class description into a design class.
   - Check each design class against design criteria; consider inheritance issues.
   - Define methods and messages associated with each design class.
   - Evaluate and select design patterns for a design class or a subsystem.
   - Review design classes and revise as required.
5. Design any interface required with external systems or devices.
6. Design the user interface:
   - Review results of task analysis.
   - Specify action sequence based on user scenarios.
   - Create a behavioral model of the interface.
   - Define interface objects and control mechanisms.
   - Review the interface design, and revise as required.
7. Conduct component-level design. Specify all algorithms at a relatively low level of abstraction.
   - Refine the interface of each component.
   - Define component-level data structures.
   - Review each component, and correct all errors uncovered.
8. Develop a deployment model.

5 Variability-intensive systems refers to systems that may be required to be self-modifying based on changes in the run-time environment or families of software products resulting from product line engineering practices for building specialized product variants out of existing software products.

9.3 Design Concepts

Several fundamental software design concepts have evolved over the history of software engineering. Although the degree of interest in these concepts has varied over the years, each has stood the test of time. Each provides the software designer with a foundation from which more sophisticated design methods can be applied. Each helps you define criteria that can be used to partition software into individual components, separate out data structure detail from a conceptual representation of the software, and establish uniform criteria that define the technical quality of a software design. These concepts help developers design the software actually needed, rather than simply focusing on creating any old working program.

9.3.1 Abstraction

When you consider a modular solution to any problem, many levels of abstraction can be posed. At the highest level of abstraction, a solution is stated in broad terms using the language of the problem environment (e.g., a user story). At lower levels of abstraction, a more detailed description of the solution is provided. Problem-oriented terminology is coupled with implementation-oriented terminology to state a solution (e.g., use case). Finally, at the lowest level of abstraction, the solution is stated in a manner that can be directly implemented (e.g., pseudocode).

As different levels of abstraction are developed, you work to create both procedural and data abstractions. A *procedural abstraction* refers to a sequence of instructions that have a specific and limited function. The name of a procedural abstraction implies these functions, but specific details are suppressed. An example of a procedural abstraction would be the word *use* for a camera in the *SafeHome* system. *Use* implies a long sequence of procedural steps (e.g., activate the *SafeHome* system on a mobile device, log on to the *SafeHome* system, select a camera to preview, locate the camera controls on mobile app user interface, etc.).[6]

A *data abstraction* is a named collection of data that describes a data object. In the context of the procedural abstraction *open,* we can define a data abstraction called **camera.** Like any data object, the data abstraction for **camera** would encompass a set of attributes that describe the camera (e.g., camera ID, location, field view, pan angle, zoom). It follows that the procedural abstraction *use* would make use of information contained in the attributes of the data abstraction **camera.**

9.3.2 Architecture

Software architecture alludes to "the overall structure of the software and the ways in which that structure provides conceptual integrity for a system" [Sha15]. In its simplest form, architecture is the structure or organization of program components (modules), the ways in which these components interact, and the structure of data that are

6 It should be noted, however, that one set of operations can be replaced with another, if the function implied by the procedural abstraction remains the same. Therefore, the steps required to implement *use* would change dramatically if the camera were automatic and attached to a sensor that automatically triggered an alert on your mobile device.

used by the components. In a broader sense, however, components can be generalized to represent major system elements and their interactions.

One goal of software design is to derive an architectural rendering of a system. This rendering serves as a framework from which more detailed design activities are conducted. A set of architectural patterns enables a software engineer to reuse design-level concepts.

Shaw and Garlan [Sha15] describe a set of properties that should be specified as part of an architectural design. *Structural properties* define "the components of a system (e.g., modules, objects, filters) and the manner in which those components are packaged and interact with one another." *Extra-functional properties* address "how the design architecture achieves requirements for performance, capacity, reliability, security, adaptability, and other system characteristics (e.g., nonfunctional system requirements)." *Families of related systems* "draw upon repeatable patterns that are commonly encountered in the design of families of similar systems."[7]

Given the specification of these properties, the architectural design can be represented using one or more of several different models [Gar95]. *Structural models* represent architecture as an organized collection of program components. *Framework models* increase the level of design abstraction by attempting to identify repeatable architectural design frameworks (patterns) that are encountered in similar types of applications. *Dynamic models* address the behavioral aspects of the program architecture, indicating how the structure or system configuration may change as a function of external events. *Process models* focus on the design of the business or technical process that the system must accommodate. Finally, *functional models* can be used to represent the functional hierarchy of a system.

Several different *architectural description languages* (ADLs) have been developed to represent these models [Sha15]. Although many different ADLs have been proposed, the majority provide mechanisms for describing system components and the ways in which they are connected to one another.

You should note that there is some debate about the role of architecture in design. Some researchers argue that the derivation of software architecture should be separated from design and occurs between requirements engineering actions and more conventional design actions. Others believe that the derivation of architecture is an integral part of the design process. The ways in which software architecture is characterized and its role in design are discussed in Chapter 10.

9.3.3 Patterns

Brad Appleton defines a *design pattern* in the following manner: "A pattern is a named nugget of insight which conveys the essence of a proven solution to a recurring problem within a certain context amidst competing concerns" [App00]. Stated in another way, a design pattern describes a design structure that solves a well-defined design problem within a specific context and amid "forces" that may have an impact on the manner in which the pattern is applied and used.

---

7 These families of related software products sharing common features are called *software product lines*.

The intent of each design pattern is to provide a description that enables a designer to determine (1) whether the pattern is applicable to the current work, (2) whether the pattern can be reused (hence, saving design time), and (3) whether the pattern can serve as a guide for developing a similar, but functionally or structurally different, pattern. Design patterns are discussed in detail in Chapter 14.

9.3.4 Separation of Concerns

Separation of concerns is a design concept [Dij82] that suggests that any complex problem can be more easily handled if it is subdivided into pieces that can each be solved and/or optimized independently. A *concern* is a feature or behavior that is specified as part of the requirements model for the software. By separating concerns into smaller and therefore more manageable pieces, a problem takes less effort and time to solve.

It follows that the perceived complexity of two problems when they are combined is often greater than the sum of the perceived complexity when each is taken separately. This leads to a divide-and-conquer strategy—it's easier to solve a complex problem when you break it into manageable pieces. This has important implications for software modularity.

Separation of concerns is manifested in other related design concepts: modularity, functional independence, and refinement. Each will be discussed in the subsections that follow.

9.3.5 Modularity

Modularity is the most common manifestation of separation of concerns. Software is divided into separately named and addressable components, sometimes called *modules,* that are integrated to satisfy problem requirements.

It has been stated that "modularity is the single attribute of software that allows a program to be intellectually manageable" [Mye78]. Monolithic software (i.e., a large program composed of a single module) cannot be easily grasped by a software engineer. The number of control paths, span of reference, number of variables, and overall complexity would make understanding close to impossible. In almost all instances, you should break the design into many modules, hoping to make understanding easier and reduce the cost required to build the software.

Recalling our discussion of separation of concerns, it is possible to conclude that if you subdivide software indefinitely the effort required to develop it will become negligibly small! Unfortunately, other forces come into play, causing this conclusion to be (sadly) invalid. Referring to Figure 9.2, the effort (cost) to develop an individual software module tends to decrease as the total number of modules increases.

Given the same set of requirements, the more modules used in your program means smaller individual sizes. However, as the number of modules grows, the effort (cost) associated with integrating modules with each other grows. These characteristics lead to a total cost or effort curve, shown in Figure 9.2. There is a number, *M,* of modules that would result in minimum development cost, but we do not have the necessary sophistication to predict *M* with assurance.

The curves shown in Figure 9.2 do provide useful qualitative guidance when modularity is considered. You should modularize, but care should be taken to stay in the vicinity of *M*. Using too few modules or too many modules should be avoided.